

```
In [6]: !pip install sqlalchemy pymysql
```

```
Collecting sqlalchemy
```

```
[notice] A new release of pip is available: 25.2 -> 26.1.1
```

```
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```

Downloading sqlalchemy-2.0.50-cp312-cp312-win_amd64.whl.metadata (9.8 kB)
Collecting pymysql
  Downloading pymysql-1.2.0-py3-none-any.whl.metadata (4.3 kB)
Collecting greenlet>=1 (from sqlalchemy)
  Downloading greenlet-3.5.1-cp312-cp312-win_amd64.whl.metadata (3.9 kB)
Requirement already satisfied: typing-extensions>=4.6.0 in c:\users\ferna\appdata\local\programs\python\python312\lib\site-packages (from sqlalchemy) (4.15.0)
Downloading sqlalchemy-2.0.50-cp312-cp312-win_amd64.whl (2.1 MB)
----- 0.0/2.1 MB ? eta -:-:-
----- 0.0/2.1 MB ? eta -:-:-
----- 0.5/2.1 MB 2.8 MB/s eta 0:00:01
----- 1.0/2.1 MB 2.6 MB/s eta 0:00:01
----- 1.6/2.1 MB 2.7 MB/s eta 0:00:01
----- 2.1/2.1 MB 2.7 MB/s 0:00:00
Downloading pymysql-1.2.0-py3-none-any.whl (45 kB)
Downloading greenlet-3.5.1-cp312-cp312-win_amd64.whl (238 kB)
Installing collected packages: pymysql, greenlet, sqlalchemy

```

```
----- 2/3 [sqlalchemy]
----- 2/3 [sqlalchemy]
----- 3/3 [sqlalchemy]
```

Successfully installed greenlet-3.5.1 pymysql-1.2.0 sqlalchemy-2.0.50

```
In [7]: from sqlalchemy import create_engine, text
import pandas as pd

engine = create_engine("mysql+pymysql://looqbox-challenge:looq-challenge@35.199.

# Ver todas as tabelas do schema
with engine.connect() as c:
    resultado = c.execute(text("SHOW TABLES"))
    for tabela in resultado:
        print(tabela)
```

```
('IMDB_movies',)
('data_product',)
('data_product_sales',)
('data_store_cad',)
('data_store_sales',)
```

```
In [8]: query1 = """
SELECT product_cod, product_name, product_val
FROM data_product
ORDER BY product_val DESC
LIMIT 10
"""

df1 = pd.read_sql(query1, engine)
print(df1)
```

	product_cod	product_name	product_val
0	301409	Whisky Escoces THE MACALLAN Ruby Garrafa 700ml...	741.99
1	176185	Whisky Escoces JOHNNIE WALKER Blue Label Garra...	735.90
2	315481	Cafeteira Espresso 3 CORACOES Tres Modo Vermelho	499.00
3	100280	Vinho Portugues Tinto Vintage QUINTA DO CRASTO...	445.90
4	320046	Escova Dental Eletrica ORAL B D34 Professional...	399.90
5	190817	Champagne Rose VEUVE CLICQUOT PONSARDIM Garraf...	366.90
6	153795	Champagne Frances Brut Imperial MOET Rose Garr...	359.90
7	311397	Conjunto de Panelas Allegra em Inox TRAMONTINA...	359.00
8	147706	Whisky Escoces CHIVAS REGAL 18 Anos Garrafa 750ml	329.90
9	154431	Champagne Frances Brut Imperial MOET & CHANDON...	315.90

```
In [9]: query2 = """
SELECT DISTINCT dep_name, section_name
FROM data_product
WHERE dep_name IN ('BEBIDAS', 'PADARIA')
ORDER BY dep_name, section_name
"""

df2 = pd.read_sql(query2, engine)
print(df2)
```

	dep_name	section_name
0	BEBIDAS	BEBIDAS
1	BEBIDAS	CERVEJAS
2	BEBIDAS	REFRESCOS
3	BEBIDAS	VINHOS
4	PADARIA	DOCES-E-SOBREMESAS
5	PADARIA	GESTANTE
6	PADARIA	PADARIA
7	PADARIA	QUEIJOS-E-FRIOS

```
In [10]: query3 = """
SELECT
    sc.business_name,
    SUM(ss.sales_value) AS total_vendas
FROM data_store_sales ss
JOIN data_store_cad sc ON ss.store_code = sc.store_code
WHERE ss.date BETWEEN '2019-01-01' AND '2019-03-31'
GROUP BY sc.business_name
ORDER BY total_vendas DESC
"""

df3 = pd.read_sql(query3, engine)
print(df3)
```

	business_name	total_vendas
0	Farma	81776691.73
1	Varejo	81032347.65
2	Atacado	80384884.60
3	Proximidade	80171122.80
4	Posto	32072326.40

```
In [11]: def retrieve_data(product_code=None, store_code=None, date=None):

    query = "SELECT * FROM data_product_sales WHERE 1=1"

    if product_code is not None:
        query += f" AND product_code = {product_code}"

    if store_code is not None:
        query += f" AND store_code = {store_code}"

    if date is not None:
        query += f" AND date BETWEEN '{date[0]}' AND '{date[1]}'"

    return pd.read_sql(query, engine)
```

```
In [12]: # Teste
df_teste = retrieve_data(product_code=None, store_code=1, date=['2019-01-01', '2019-01-05'])
print(df_teste.head())
```

	STORE_CODE	PRODUCT_CODE	DATE	SALES_VALUE	SALES_QTY
0	1	18	2019-01-01	708.5	65.0
1	1	18	2019-01-02	1297.1	119.0
2	1	18	2019-01-03	1144.5	105.0
3	1	18	2019-01-04	1090.0	100.0
4	1	18	2019-01-05	893.8	82.0

```
In [14]: query_lojas = """
SELECT
    STORE_CODE,
    STORE_NAME,
```

```

        START_DATE,
        END_DATE,
        BUSINESS_NAME,
        BUSINESS_CODE
    FROM data_store_cad
    """

query_vendas = """
SELECT
    STORE_CODE,
    DATE,
    SALES_VALUE,
    SALES_QTY
FROM data_store_sales
WHERE DATE BETWEEN '2019-01-01' AND '2019-12-31'
"""

df_lojas = pd.read_sql(query_lojas, engine)
df_vendas = pd.read_sql(query_vendas, engine)

# Converter DATE para datetime e filtrar
df_vendas['DATE'] = pd.to_datetime(df_vendas['DATE'])
df_vendas = df_vendas[
    (df_vendas['DATE'] >= '2019-10-01') &
    (df_vendas['DATE'] <= '2019-12-31')
]

# Juntar as duas tabelas
df_merged = df_vendas.merge(df_lojas, on='STORE_CODE')

# Calcular TM por Loja
df_tm = df_merged.groupby(['STORE_NAME', 'BUSINESS_NAME']).agg(
    total_valor=('SALES_VALUE', 'sum'),
    total_qty=('SALES_QTY', 'sum')
).reset_index()

df_tm['TM'] = (df_tm['total_valor'] / df_tm['total_qty']).round(2)

# Tabela final
df_resultado = df_tm[['STORE_NAME', 'BUSINESS_NAME', 'TM']].rename(columns={
    'STORE_NAME': 'Loja',
    'BUSINESS_NAME': 'Categoria',
})

print(df_resultado.sort_values('Loja').to_string(index=False))

```

	Loja	Categoria	TM
	Bahia	Atacado	15.39
	Bangkok	Posto	13.67
	Belem	Proximidade	15.37
	Berlin	Proximidade	15.39
	Buenos Aires	Atacado	15.39
	Chicago	Varejo	15.53
	Dubai	Atacado	15.39
	Hong Kong	Farma	26.35
	London	Farma	28.99
	Madri	Farma	29.03
	Miami	Posto	13.67
	New York	Proximidade	15.39
	Paris	Proximidade	15.39
	Rio de Janeiro	Farma	29.59
	Roma	Varejo	15.39
	Salvador	Atacado	15.39
	Sao Paulo	Varejo	15.39
	Sidney	Posto	13.67
	Tokio	Varejo	15.39
	Vancouver	Posto	13.67

```
In [15]: df_imdb = pd.read_sql("SELECT * FROM IMDB_movies LIMIT 5", engine)
print(df_imdb.columns.tolist())
print(df_imdb.head())
```

```
['Id', 'Title', 'Genre', 'Director', 'Actors', 'Year', 'Runtime', 'Rating', 'Vote
s', 'RevenueMillions', 'Metascore']
```

	Id	Title	Genre \
0	1	Guardians of the Galaxy	Action,Adventure,Sci-Fi
1	2	Prometheus	Adventure,Mystery,Sci-Fi
2	3	Split	Horror,Thriller
3	4	Sing	Animation,Comedy,Family
4	5	Suicide Squad	Action,Adventure,Fantasy

	Director	Actors \
0	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...
1	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...
2	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...
3	Christophe Lourdelet	Matthew McConaughey, Reese Witherspoon, Seth Ma...
4	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...

	Year	Runtime	Rating	Votes	RevenueMillions	Metascore
0	2014	121	8.0	757074	333.0	76
1	2012	124	7.0	485820	126.0	65
2	2016	117	7.0	157606	138.0	62
3	2016	108	7.0	60545	270.0	59
4	2016	123	6.0	393727	325.0	40

```
In [16]: import matplotlib.pyplot as plt

df_imdb = pd.read_sql("SELECT * FROM IMDB_movies", engine)

# Pegar só o primeiro gênero de cada filme
df_imdb['GenrePrimary'] = df_imdb['Genre'].str.split(',').str[0]

# Rating médio por gênero
df_genre = df_imdb.groupby('GenrePrimary')['Rating'].mean().round(2).sort_values

# Gráfico
```

```

plt.figure(figsize=(12, 6))
bars = plt.bar(df_genre.index, df_genre.values, color='steelblue')
plt.title('Rating Médio por Gênero (IMDB)', fontsize=14)
plt.xlabel('Gênero')
plt.ylabel('Rating Médio')
plt.xticks(rotation=45, ha='right')

# Mostrar valor em cima de cada barra
for bar, val in zip(bars, df_genre.values):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.05,
             str(val), ha='center', fontsize=9)

plt.tight_layout()
plt.savefig('grafico_imdb.png', dpi=150)
plt.show()

```

